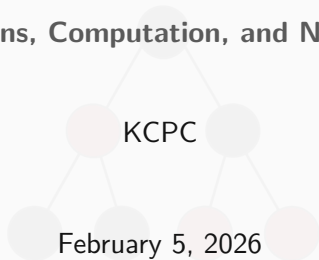
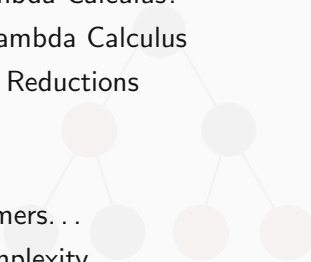


The Lambda Calculus

Functions, Computation, and Numbers



Overview

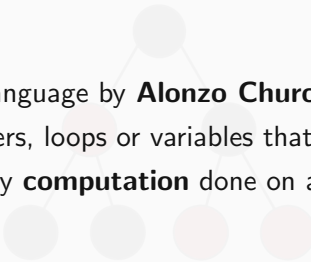
- 
- 1 What is the Lambda Calculus?
 - 2 Syntax of the Lambda Calculus
 - 3 Alpha and Beta Reductions
 - 4 Currying
 - 5 Some Purity
 - 6 We're Programmers. . .
 - 7 Some More Complexity
 - 8 Conclusion

What is the Lambda Calculus?



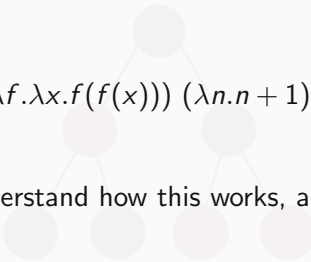
What is the Lambda Calculus?

Overview

- 
- Mathematical language by **Alonzo Church** in the 1930s
 - It has no numbers, loops or variables that store values
 - Can describe any **computation** done on a computer

What is the Lambda Calculus?

Example


$$(\lambda f. \lambda x. f(f(x))) (\lambda n. n + 1) 0$$

Evaluates to: 2

Goal of today: Understand how this works, and creating our own mathematics!

Syntax of the Lambda Calculus



What Is a Function?

- A function takes an **input**
- And produces an **output**

Mental model: A function is a machine that takes an input and produces an output.

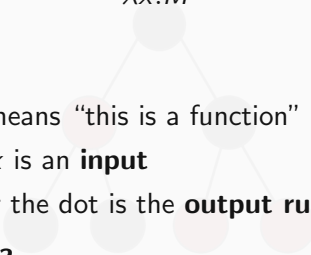
Important restriction:

- In the lambda calculus, a function takes **one input only**

Building a Function

In the lambda calculus, we build functions using **abstraction**.

$\lambda x.M$



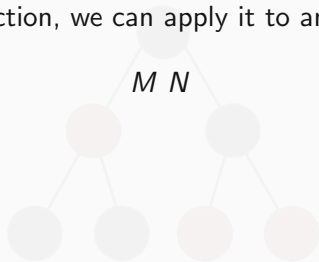
- The symbol λ means “this is a function”
- It tells us that x is an **input**
- Everything after the dot is the **output rule**

Why do we need λ ?

- To clearly say when a variable is an input
- To avoid confusion between inputs and ordinary names
- To make functions precise and unambiguous

Applying a Function

Once we have a function, we can apply it to an input.



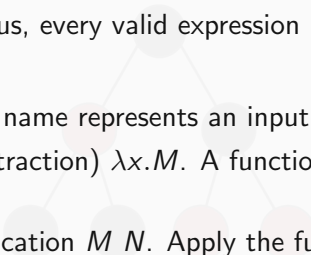
→ M is a function

→ N is its input

→ Application means “run the machine”

The Three Building Blocks

In the lambda calculus, every valid expression is called a **term**. A term is

- 
- 1 A variable x . A name represents an input.
 - 2 A function (abstraction) $\lambda x.M$. A function that takes input x and returns M .
 - 3 A function application $M N$. Apply the function M to the argument N .

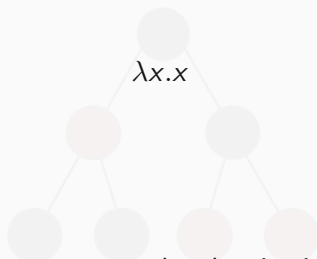
Syntax of the Lambda Calculus

Example: Identity Function

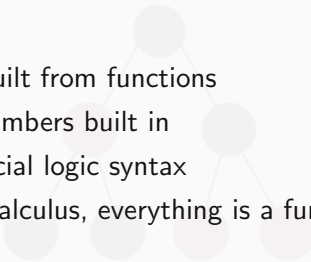
→ Input: x

→ Output: x

→ **Interpretation:** return exactly what is given



Functions, Functions Everywhere

- 
- Every term is built from functions
 - There are no numbers built in
 - There is no special logic syntax
 - In the lambda calculus, everything is a function

Alpha and Beta Reductions



Alpha Conversion

Alpha conversion is:

- The process of renaming a **bound** variable
- Used to avoid confusion or name clashes

General form:

$$\lambda x.M \longrightarrow_{\alpha} \lambda y.M[y/x]$$

Important rule:

- Renaming must not change how the input is used; choose y fresh.

Alpha Equivalence

Alpha equivalence means:

- Two functions are considered the same
- If they differ only by input names

Example: $\lambda x.x \equiv_{\alpha} \lambda y.y$

Key idea:

- Variable names are just labels
- What matters is how inputs are used

Beta Reduction

Beta reduction is where computation happens.

Applying a function means: *Replace the input variable with the argument.*

General rule:

$$(\lambda x.M) N \longrightarrow_{\beta} M[N/x]$$

Example: $(\lambda x.x + 1) 3 \longrightarrow 3 + 1 \longrightarrow 4.$

Substitute only free occurrences of x ; rename binders first if needed.

Currying



Functions Take One Input

- In the lambda calculus, every function takes **exactly one input**
- There are no two-input or three-input functions

Problem:

- Many things in maths need more than one input
- For example: addition, multiplication

The Idea Behind Currying

Solution: A function can return another function.

- First input is given
- A new function is returned
- That new function waits for the next input

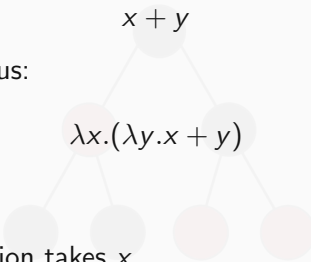
This idea is called **currying**.

Currying Explained with Addition

In ordinary maths:

$$x + y$$

In the lambda calculus:

$$\lambda x. (\lambda y. x + y)$$


- The outer function takes x
- It returns a new function
- The inner function takes y

Currying in Action

First apply the function to 1:

$$(\lambda x. \lambda y. x + y) 1 \longrightarrow \lambda y. 1 + y$$

Then apply the result to 2:

$$(\lambda y. 1 + y) 2 \longrightarrow 3$$

Conclusion:

- One input at a time
- But still behaves like a two-input function

Some Purity



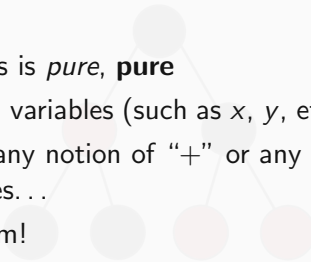
Small Issue

So far, we've used some notation without actually defining them. . .

→ $\lambda x.(\lambda y.x + y)$: what is a “+”?

→ $(\lambda y.1 + y) 2 \rightarrow 3$: what is a “2”? what is a “number”?

Purest of all Purest

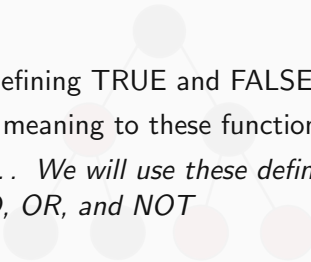
- 
- Lambda calculus is *pure*, **pure**
 - We only have λ , variables (such as x , y , etc.), and “(” and “)”
 - We don't have any notion of “+” or any of the numbers, or even truth values. . .
 - Let's create them!

We're Programmers...



We're Programmers. . .

. . . Bits!

- 
- Let's begin by defining TRUE and FALSE
 - Let's give some meaning to these functions
 - *Foreshadowing. . . We will use these definitions to create the operations AND, OR, and NOT*

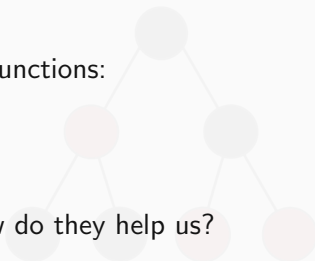
Some Hint?

Think of these two functions:

→ $\lambda x. \lambda y. x$

→ $\lambda x. \lambda y. y$

What are they? How do they help us?



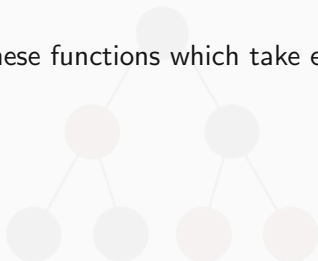
We Orn't And

Let's try to define these functions which take either a single or two parameters:

→ AND: (x, y)

→ OR: (x, y)

→ NOT: (x)



Some More Complexity

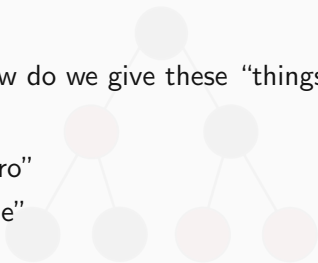


Unnaturally Natural Numbers

0, 1, 2, 3, 4, 5, ... How do we give these “things” meaning via lambda calculus?

→ Let's define “zero”

→ Let's define “one”



Oh, I Know This!

We can use *induction* to continue this pattern and to define all natural numbers. Find the successor function for your numbering system:

$$\text{SUCC}(n) = n + 1$$

Finally Some Mathematics

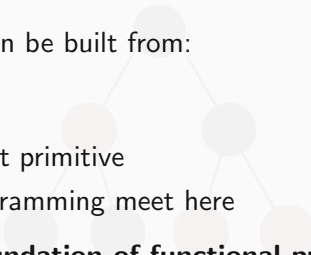
Using what we discovered and learned so far, let's finalise "our" mathematics:

- ADD: $(x, y) \mapsto x + y$
- SUB: $(x, y) \mapsto x - y$
- MUL: $(x, y) \mapsto x \times y$
- POW: $(x, y) \mapsto x^y$
- DIV: $(x, y) \mapsto x/y$ (if you can do this without looking it up, I'll give you a candy)

Conclusion



What You've Learned

- 
- Computation can be built from:
 - Functions
 - Substitution
 - Numbers are not primitive
 - Maths and programming meet here

This is also the foundation of functional programming

Conclusion

Try It Yourself: Booleans (Answers)

$T = \lambda x.\lambda y.x$, $F = \lambda x.\lambda y.y$; $T\ a\ b \rightarrow_{\beta} a$ and $F\ a\ b \rightarrow_{\beta} b$.

$AND = \lambda p.\lambda q.p\ q\ F$,

$OR = \lambda p.\lambda q.p\ T\ q$,

$NOT = \lambda p.p\ F\ T$.

For example, $AND\ T\ F \rightarrow_{\beta} T\ F\ F \rightarrow_{\beta} F$.

Try It Yourself: Numbers (Answers)

A Church numeral $\bar{n}$ applies a function n times:

$$\bar{0} = \lambda f.\lambda x.x, \quad \bar{1} = \lambda f.\lambda x.fx, \quad \bar{2} = \lambda f.\lambda x.f(fx).$$

$$\text{SUCC} = \lambda n.\lambda f.\lambda x.f(nfx)$$

Thus $\text{SUCC } \bar{n} f x \rightarrow_{\beta} f(nfx)$, which applies f exactly $n + 1$ times.

Try It Yourself: Arithmetic (Answers)

$$\text{ADD} = \lambda m. \lambda n. \lambda f. \lambda x. mf(nfx),$$

$$\text{MUL} = \lambda m. \lambda n. \lambda f. m(nf),$$

$$\text{POW} = \lambda m. \lambda n. \lambda f. \lambda x. nmfx.$$

They perform $m + n$, mn , and m^n iterations respectively. SUB is truncated at zero for Church naturals; DIV additionally requires a zero-divisor convention.



Conclusion

Try It Yourself: Subtraction and Division (Answers)

Let $\langle a, b \rangle = \lambda s. sab$ and $\text{STEP} \langle a, b \rangle = \langle b, \text{SUCC } b \rangle$. Iterating STEP n times from $\langle 0, 0 \rangle$ leaves $\langle \max(n - 1, 0), n \rangle$. Hence

$$\text{PRED} = \lambda n. \text{FIRST}(n \text{STEP} \langle 0, 0 \rangle), \quad \text{SUB} = \lambda m. \lambda n. n \text{PRED } m.$$

For $d > 0$, DIV iterates m times on a pair (q, r) initially $(0, 0)$: increment r ; if $r = d$, reset it to 0 and increment q . The invariant after k steps is $k = qd + r$, $0 \leq r < d$; the final q is $\lfloor m/d \rfloor$. Set $\text{DIV}(m, 0) = 0$ by convention. See the full lambda-term construction in the solutions sheet.

